

# Guía FP Ejercicio

## Numero negativo, positivo o cero

**Propósito de la guía.** Acompañarte paso a paso para desarrollar el ejercicio de números, primero en PSeInt y luego en Code::Blocks, cuidando la lógica, la sintaxis y la prueba de escritorio. La idea es que no solo funcione, sino que entiendas por qué funciona.

### Resultado esperado

Leer un número real ingresado por teclado y determinar si es: positivo, negativo, o igual a cero.

## Contexto del ejercicio

En este ejercicio el programa solicita un número real al usuario.

Luego, mediante estructuras condicionales, analiza el valor ingresado para determinar si el número es mayor que cero, menor que cero o exactamente igual a cero.

Las condiciones utilizadas son:

$x > 0$

$x < 0$

$x = 0$

## 1. Seudocódigo para PSeInt

Antes de programar en C, conviene escribir la solución en PSeInt. Aquí se observa la lógica con palabras cercanas al lenguaje natural.

Proceso NumeroPositivoNegativoCero

Definir x Como Real

Escribir "Ingresar un numero: "

Leer x

Si  $x > 0$  Entonces

Escribir "El numero es positivo"

SiNo

Si  $x < 0$  Entonces

Escribir "El numero es negativo"

SiNo

Escribir "El numero es cero"

FinSi

FinSi

FinProceso

Tip amigable: Imagina que si ninguna condición se cumple, entonces el numero debe de ser cero.

## 2. Prueba de escritorio en PSeInt

La prueba de escritorio permite comprobar manualmente qué hace el algoritmo. Para que sea fácil de revisar, se muestra el comportamiento.

| X  | Resultado |
|----|-----------|
| 5  | Positivo  |
| -3 | Negativo  |
| 0  | Cero      |

Salida esperada:

Ingresar un número: 5  
El número es positivo

Ingresar un número: -3  
El número es negativo

Ingresar un número: 0  
El número es cero

## 3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

| PSeInt                 | C / Code::Blocks             | Para recordar                                   |
|------------------------|------------------------------|-------------------------------------------------|
| Proceso ... FinProceso | int main() { ... return 0; } | Todo programa en C inicia en main.              |
| Definir x como real    | Float x;                     | Los números reales se declaran con float.       |
| Escribir               | Printf();                    | Printf muestra mensajes y resultados.           |
| Leer x                 | Scanf("%f", "%x");           | Scanf permite ingresar datos                    |
| Si ... Entonces        | If (.)                       | If evalúa condiciones                           |
| Sino                   | Else                         | Else se ejecuta cuando la condición es falsa.   |
| X > 0                  | X > 0                        | Los operadores relacionales se mantienen igual. |

## Código validado en C para Code::Blocks

```
#include <stdio.h>

int main()
{
    float x;

    printf("Ingresar un numero: ");
    scanf("%f", &x);

    if(x > 0)
    {
        printf("El numero es positivo");
    }
    else
    {
        if(x < 0)
        {
            printf("El numero es negativo");
        }
        else
        {
            printf("El numero es cero");
        }
    }

    return 0;
}
```

## Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.
5. Probar valores positivos, negativos y cero.
6. Comparar la salida con la prueba de escritorio.

Errores comunes que debes evitar:

- Usar = en lugar de operadores relacionales.
- Olvidar el símbolo & en scanf.
- Confundir if con else.
- Declarar variables reales como int.
- No cerrar correctamente las llaves {}.

## 4. Rúbrica de evaluación sobre 20 puntos

Esta rúbrica permite valorar el desarrollo completo del ejercicio, desde la comprensión del problema hasta la ejecución en Code::Blocks.

| Criterio                       | Excelente                                                     | Bueno                                   | En proceso                       | Puntaje |
|--------------------------------|---------------------------------------------------------------|-----------------------------------------|----------------------------------|---------|
| Comprensión del problema       | Identifica correctamente números positivos, negativos y cero. | Comprende parcialmente las condiciones. | Confunde las comparaciones.      | 4 pts   |
| Seudocódigo en PSeInt          | Usa correctamente estructuras condicionales.                  | Presenta pequeños errores de sintaxis.  | El algoritmo está incompleto.    | 4 pts   |
| Prueba de escritorio           | Comprueba correctamente los resultados.                       | Presenta una prueba parcial.            | No evidencia seguimiento lógico. | 4 pts   |
| Traducción a C en Code::Blocks | El código compila y ejecuta correctamente.                    | Tiene errores menores corregibles.      | El código no compila o falla.    | 5 pts   |
| Presentación y orden           | Trabajo limpio y organizado.                                  | Comprensible con pequeños detalles.     | Trabajo desordenado.             | 3 pts   |

**Total: 20 puntos.** Para una buena calificación, lo más importante es que exista coherencia entre pseudocódigo, prueba de escritorio, código en C y salida final.

### Lista rápida de verificación antes de entregar

- El programa solicita un número.
- El dato ingresado se almacena correctamente.
- Se usan estructuras condicionales correctamente.
- El programa identifica positivos, negativos y cero.
- El resultado se muestra correctamente.
- El código compila sin errores.